

Education

Computer Science—A Vicious Circle

Abraham Kandel

*New Mexico Institute of Mining and Technology**

Key Words and Phrases: education, computer engineering, computer science curriculum, systems design

CR Categories: 1.52, 6.0

In many computer science departments throughout the country, including some interdisciplinary departments, the curriculum has placed increasing emphasis upon applied mathematics and the fundamental nature of computational systems. A set of core courses which usually are fitted into such a program can be found in the report of the ACM curriculum committee on computer science, Curriculum 68 [1]. Some computer science departments have done such a magnificent job of de-emphasizing the importance of the experimental laboratory in their program that their graduates emerge thoroughly unprepared to tackle the intricacies associated with design work in the real-life world—both in software and hardware. This is basically because most computer science departments do not emphasize sufficiently the practical aspects of computational system design and implementation. For example, some students have serious difficulty in designing even a shift register of an 8-4-2-1 counter with ripple carry, not to mention more complicated computer systems. Perhaps even more important, computer science in the academic sense has implemented the “modus ponens” (rule of detachment) from the real-life world and has very little to do with real computers. The relationship between the abstract side of computer science and questions of cost and marketing are hardly ever mentioned in courses.

The increasing gap between the abstract and applied aspects of computer science turns the problem of

applying theory to real-life projects into a very serious one in comparison with the same problem 20 or even 10 years ago.

No doubt there is much beauty in an abstract approach to computer science. Eventually, we hope, this approach will benefit the computer industry and, ultimately, our society. However, we must take the right steps in training computer science students to benefit the industry *now*. This can easily be accomplished by combining theoretical courses with a practical approach to *computer engineering*.

Industry gets graduates from computer science departments with a bag full of the latest technical jargon but no depth of understanding of real computer systems and with no concept of the problems they will be asked to face and solve. Computer science graduates have programmed only a limited set of problems in computer languages, many of which they have never finished or tested adequately. The average graduate has no solid idea of what happens when design projects get large, as most real-life problems do. In this category we include large systems design (hardware and software), file management and efficient implementation of complex programs which exceed available core storage or take a long time to execute. Most of these graduates are not well prepared to fill the needs of the computer industries, business, and governmental agencies, because their background as computer science majors is generally inadequate for this purpose.

It is quite obvious that there is no effective theory of computer science as such. In fact, there are no effective models of computers. However some theories seem to fit naturally into the scope of the theoretical aspects of computer science. For example, there are the theories of automata, formal languages, computational complexity, combinatorics, systems, applied abstract algebra (Boolean algebra, lattices, semigroups, universal algebra), metamathematics, coding, and algorithm theory. Usually courses are approached through theory and models. However, in several cases (courses in computer system organization or computer design), a case-study approach on a large-scale project would help the student to understand the formalism of the subject and to see precisely its entire practical content. Case studies provide examples of good standard practice and documentation and reflect modern design features. (By “design features” we mean hardware and software design, as it is our purpose not to separate these two aspects.)

Most courses attempt to satisfy the following general goals: (a) develop a solid design methodology; (b) present the relevant key ideas and their implementation;

Copyright © 1972, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted, provided that reference is made to this publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

* Computer Science Department, Socorro, NM 87801.

and (c) develop familiarity with the basic techniques of the topic under discussion.

However, most of them do not attempt to gain experience in grappling with life-like, large complex systems. It is true that computer science students may perform experimental work in hardware design, with algorithms, programs, and applications. But it should be clear that this is not enough; the student should also achieve real understanding of the inherent problems and their implications. Only then can he try to utilize his knowledge by looking at the details of the techniques used to implement his system. The system should be a real working one, in which good standard practice and state-of-the-art techniques should be used. The first step in achieving this is to change the dialogue between the commercial user of computer science personnel and the computer science educator from an imitation into a practical path. It is important that the vicious circle between computer science in the academic environment and the computer industry should be closed.

Even though much of the complexity of modern computer systems is due to the need for efficiency in the design, we should teach the prospective designer not to worry about how well the computational system works before determining whether it will work at all. Then, consideration of efficiency and other real-life constraints (cost, marketing, etc.) must be emphasized until a real-life design is achieved by the gradual removal of the unrealistic assumptions which build the first layer in the student's studies. In order to reinforce this development and to lend credibility to the system design (hardware or software), the student's work must be complemented by a practical, carefully-designed case study. One attempt to go quite far in this direction has been made in the design of the UHC-1 at the University of Hawaii [2]. As pointed out in the report on the UHC-1, it has been possible for a group of students to design something meaningful with emphasis on doing rather than on learning how to do.

Therefore, it is recommended that design projects, such as the minicomputer UHC-1, from basic specifications to complete circuitry and machine language, be performed in a senior-level course on computer design in order to stimulate student interest in computer-based industries, to increase communication between groups of students on a design team (after all, industry is based on a team effort), and to expose the students to real-life constraints (cost is "sometimes" a prime consideration).

Conclusion

1. Since computer science is an applied science, it seems wise that computer science departments maintain a close contact with computer-based industries.

2. The basic difference between stimulating student interest toward graduate work or toward work in the computer industry should be taken into account when

a computer science curriculum is designed by a department.

3. The industry requires not only people who can fill jobs that have sophisticated requirements but also people who have the know-how to implement their background in the most practical way.

4. The amalgamation of theory and the practical approach provides the computer science department with a means of access to a wide variety of languages, techniques, and systems without major capital and operating expenditures.

Received August 1971; revised September 1971

References

1. Curriculum 68. *Comm. ACM* 11, 3 (Mar. 1968), 151-197.
2. Kuo, F.F. A case study on a student-designed digital computer. Tech. Rep. A71-5. The Aloha System., U. of Hawaii, Jan. 1971.

Editorial Comments by Peter Wegner

Professor Kandel expresses the widely felt need among computer professionals for a greater practical component to computer science education. However, the inadequacy of practical training should not be blamed completely on theoreticians. The development of sound practical training programs appears to be much harder than the development of theoretical courses or even of core courses such as those in Curriculum 68 [1]. Practical training programs require a great deal of instructor's time and computer time. Moreover we do not know how to teach "good" programming habits. It is clear that we are not currently providing an optimal computer science education, either for practical programmers or for theorists. The reasons for this may, however, be more subtle than an overemphasis of theory. A number of computer science departments have devoted considerable effort to practical training programs, both internally and in cooperation with industry, but no program yet developed has been implemented by a large number of computer science departments. We may have been teaching the wrong *kind* of theory rather than too much theory. The right kind of theory is likely to have considerable relevance, just as in the case of engineering. Our problem is that we have not yet discovered the right kind of theoretical or practical basis for training computer scientists. When we do, it may well turn out that the theoretical and practical components of a computer science education will be more closely interrelated than they are at present. Further discussion of the questions raised by Professor Kandel is invited.